



United International University (UIU)
Dept. of Computer Science & Engineering (CSE)

Final Exam, Trimester: Spring 2025

Course Code: CSE 1115, Course Title: Object Oriented Programming

Total Marks: 40, Duration: 2 Hours

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

Answer all 5 Questions. Numbers on the right denote marks.

QUESTION 1

[4 + 6 = 10 Marks]

a)

[4 Marks]

Find and correct the errors. The file `Shape.java` below is meant to declare an abstract class `Shape` that implements the `Comparable` interface. It contains **three** errors. For each error, state what is wrong and write the corrected line.

`Shape.java`

```
public abstract class Shape implements Comparable<Shape> {
    protected String name;

    public Shape(String name) {
        this.name = name;
    }

    // ERROR 1: an abstract method must not have a body
    public abstract double area() {
        return 0.0;
    }

    // ERROR 2: a method declaration is missing its parentheses
    abstract void printName;

    // ERROR 3: an overriding method cannot reduce visibility
    int compareTo(Shape other) {
        return Double.compare(area(), other.area());
    }
}
```

```
public abstract class Shape implements Comparable<Shape>  protected String name;
public Shape(String name)  this.name = name;
// ERROR 1: an abstract method must not have a body public abstract double area()  return 0.0;
// ERROR 2: a method declaration is missing its parentheses abstract void printName;
// ERROR 3: an overriding method cannot reduce visibility int compareTo(Shape other)  return
Double.compare(area(), other.area());
```

b)

[6 Marks]

The interface `Operation` and its implementation `Calculator` are given on the left. Complete the code on the right so that the program prints the result of `apply(8, 12)`.

Given Code	Complete the Code
<pre>interface Operation { int apply(int a, int b); } class Calculator implements Operation { @Override public int apply(int a, int b) { return a + b; } }</pre>	<pre>public class Main { public static void main(String[] args) { // 1. Declare an Operation reference // that // points to a new Calculator // object. // 2. Call apply(8, 12) and print // the result. // // WRITE YOUR CODE HERE } }</pre>

When completed correctly, the program must produce the console output shown below.

Sample Output

> Result: 20

QUESTION 2

[3 + 3 + 4 = 10 Marks]

A bank account starts with a balance of 5000.0. Complete the skeleton program `Bank.java` below so that a withdrawal larger than the current balance raises a user-defined exception, and every transaction is reported as finished.

`Bank.java`

```
// Bank.java -- complete the three marked sections
class InsufficientFundsException extends Exception {
    // ----- (a) Write your code here -----
}

public class Bank {
    private double balance;

    public Bank(double initialBalance) {
        this.balance = initialBalance;
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        // ----- (b) Write your code here -----
    }

    public static void main(String[] args) {
        // ----- (c) Write your code here -----
    }
}
```

```
// Bank.java – complete the three marked sections
class InsufficientFundsException extends Exception
// ----- (a) Write your code here -----
public class Bank { private double balance;
public Bank(double initialBalance) { this.balance = initialBalance;
public void withdraw(double amount) throws InsufficientFundsException // ----- (b) Write
your code here -----
public static void main(String[] args) // ----- (c) Write your code here -----
```

a) [3 Marks]

Write the `InsufficientFundsException` class. It must extend `Exception` and store a descriptive message that can later be read with `getMessage()`.

b) [3 Marks]

Complete the `withdraw` method. If `amount` exceeds the current balance, throw `InsufficientFundsException` with the message “Withdrawal amount exceeds current balance.” Otherwise, deduct `amount` from the balance.

c) [4 Marks]

Complete the `main` method. Create a `Bank` object with balance 5000.0, call `withdraw(2000.0)` and `withdraw(7000.0)` inside a `try` block, catch the custom exception and print its message, and use a `finally` block that always prints “Transaction processing completed.”

Trace the program for the two calls and complete the table below.

Sample Input	Output
withdraw(2000.0) from a balance of 5000.0	Withdrawal successful. Remaining balance: 3000.0. Transaction processing completed.
withdraw(7000.0) from a balance of 5000.0	InsufficientFundsException: Withdrawal amount exceeds current balance. Transaction processing completed.

QUESTION 3

[5 Marks]

You are given a class `Product` that models a store item. Inside the `main` method of `Inventory`, complete the five tasks below using the Java Collections Framework. At least one `ArrayList` must be used.

```
Inventory.java

class Product {
    private String name;
    private double unitPrice;
    private int quantity;

    public Product(String name, double unitPrice, int quantity) {
        this.name = name;
        this.unitPrice = unitPrice;
        this.quantity = quantity;
    }

    public String getName() { return name; }
    public double getUnitPrice() { return unitPrice; }
    public int getQuantity() { return quantity; }

    public double totalPrice() {
        return unitPrice * quantity;
    }

    @Override
    public String toString() {
        return name + " (" + quantity + " @ " + unitPrice + ")";
    }
}

public class Inventory {
    public static void main(String[] args) {
        // ----- Complete tasks (i) to (v) below -----
    }
}

-----
class Product private String name; private double unitPrice; private int quantity;
public Product(String name, double unitPrice, int quantity) this.name = name; this.unitPrice =
unitPrice; this.quantity = quantity;
public String getName() return name; public double getUnitPrice() return unitPrice; public int
getQuantity() return quantity;
public double totalPrice() return unitPrice * quantity;
@Override public String toString() return name + " (" + quantity + " @ " + unitPrice + ")";
public class Inventory public static void main(String[] args) // ----- Complete tasks (i) to (v)
below -----
```

Tasks:

- (i) Declare an `ArrayList<Product>` named `products`. [1]
- (ii) Insert three products: "Pen", 5.0, 100, "Notebook", 40.0, 25 and "Eraser", 3.0, 200. [1]
- (iii) Sort `products` in descending order of `totalPrice()`. [1]
- (iv) Remove every product whose quantity is less than 30. [1]
- (v) Print each remaining product, then print the total inventory value (the sum of all `totalPrice()` values). [1]

QUESTION 4

[5 Marks]

The text file `src/transactions.txt` contains several lines, one purchase per line. Each line stores a customer ID (string), the customer name (string), and the purchase amount (double), separated by a single space. A sample input is shown in the left column below.

Write a Java program that:

- reads `src/transactions.txt` line by line, [1]
- writes every customer ID and name to `src/customers.txt`, [1]
- writes the customer name and amount to `src/summary.txt`, but only for purchases above 5000.0, [2]
- prints “Processed N records.” once all records are read. [1]

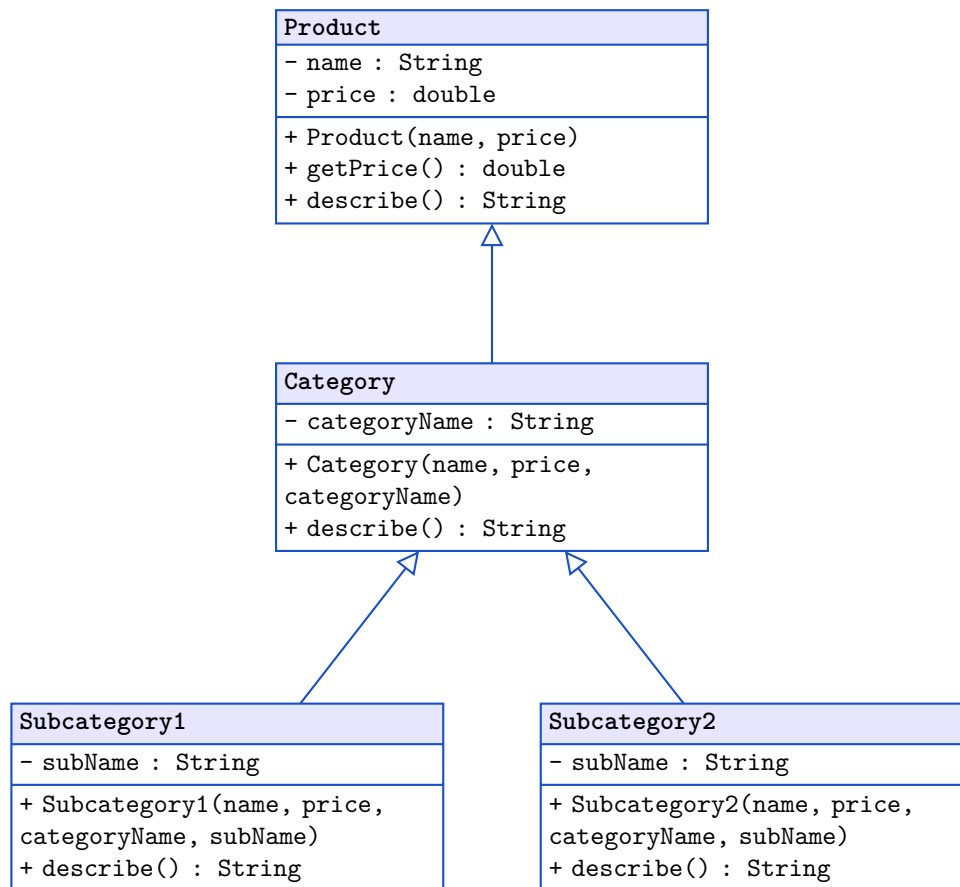
For the sample input below, your program must produce the output file shown on the right (purchases that are skipped appear as —).

<code>transactions.txt</code> (input)	<code>summary.txt</code> (output)
C-101 Ayesha 2400.0	---
C-102 Tanvir 12400.0	Tanvir 12400.0
C-103 Rakib 8700.0	Rakib 8700.0
C-104 Samia 15600.0	Samia 15600.0
C-105 Nadia 3400.0	---
C-106 Farhan 1900.0	---

QUESTION 5

[6 + 4 = 10 Marks]

The UML diagram below shows a three-level inheritance hierarchy for a small shop. Implement the hierarchy in Java and complete the tasks that follow. Coordinates given to `\umlclass` are in cm, measured from the origin of the diagram.



a) [6 Marks]

Implement `Product`, `Category`, `Subcategory1` and `Subcategory2` exactly as shown in the diagram. Every subclass must call its parent constructor with `super(...)` and override `describe()` so that it appends its own field to the parent's description.

b) [4 Marks]

Write a `main` method that creates one object of each class, stores all four in a `Product[]` array, and prints the result of `describe()` for each element. The expected console output is shown below.

Sample Output

```
> Pen (price: 5.0)
> Pen (price: 5.0) | Category: Stationery
> Pen (price: 5.0) | Category: Stationery | Sub: Writing
> Pen (price: 5.0) | Category: Stationery | Sub: Packing
```